Strategic Voting in the 2022 French Presidential Election

An Agent-Based Simulation and Deep Learning Surrogate Model

Moeez Ahmed
moeez5922@gmail.com
moeez.ahmed@ut-capitole.fr
University Toulouse Capitole
Toulouse, France

Zain Noman Farooq
zain-noman.farooq@ut-capitole.fr
University Toulouse Capitole
Toulouse, France

Pamit Duggal
pamitduggalfr@gmail.com
pamit.duggal@ut-capitole.fr
University Toulouse Capitole
Toulouse, France

Abdullah Tariq
info.abdullahtariq@gmail.com
abdullah.tariq@ut-capitole.fr
University Toulouse Capitole
Toulouse, France

Abstract

This study looks at how people vote strategically in the 2022 French presidential election. A lot of voters do not just pick their favourite they think about who can realistically win and adjust their vote to avoid wasting it. We simulated this behaviour using GAMA, where each voter is an agent connected to others on a social network. Over 60 days, every agent gets a daily poll from their neighbours and sometimes switches who they plan to vote for. Once we had enough simulation data, we trained an LSTM model to predict how voting patterns evolve over time, so we could skip running the full simulation for large populations. This paper walks through the game theory behind the decision, how we set up the simulation, how the neural network was built, and what we found when comparing both approaches.

1 Introduction

Think about the last time you heard someone say "I like candidate X but I'll vote for Y because X has no chance." That is strategic voting, and it happens all the time. People weigh not just who they prefer, but who they think can actually win. In a two-round election like France's, this calculus matters a lot. Only the top two candidates from the first round survive, so if your favourite is polling at 3%, voting for them might just hand the win to whoever you dislike the most.

The 2022 French presidential election is a perfect case to study this. There were 12 candidates on the ballot: Macron, Le Pen, Mélenchon, Zemmour, Pécresse, Jadot, Lassalle, Roussel, Dupont-Aignan, Hidalgo, Poutou, and Arthaud. Most had no real shot at the top two. So for many voters, the real question was not "who do I like most" but "who should I vote for so that the second round goes the way I want."

We tackled this problem from two angles. First, we built an agent-based simulation in GAMA where thousands of voters interact on a social network over 60 days, updating their vote based on daily polls from their connections. Second, we trained an LSTM neural network on the simulation output, so that instead of running hundreds of expensive simulations, we can get a fast prediction of how voting patterns evolve. The two approaches are quite different in how they work, and part of what we found is that each one has its place.

The remainder of this paper is structured as follows: Section 2 establishes the game-theoretic foundations behind strategic voting and defines our three agent types. Section 3 details the simulation setup and macroscopic findings. Section 4 introduces the deep learning surrogate model architecture and evaluates its predictive performance. Finally, Section 5 compares the computational trade-offs of both approaches.

2 Game-Theoretic Analysis of Strategic Voting

2.1 The Setup

Before jumping into the full simulation, we worked through a simple example to understand the logic of strategic voting. Imagine just 3 voters and 3 candidates: A, B, and C. Voter 1 is stubborn — they always vote A, no matter what anyone else does. So really the interesting question is what Voter 2 and Voter 3 should do.

Here is what each of them wants:

- Voter 2 ranks candidates as C first, then B, then A. They get 2 points if C wins, 1 point if B wins, and nothing if A wins.
- Voter 3 ranks them B first, then A, then C. They get 2 points if B wins, 1 if A wins, 0 if C wins.

Since the election has two rounds, the utility each voter gets depends on who wins the *final* round, not just who made it through. So we need to work out what happens in round two for each possible combination of first-round votes.

2.2 Second-Round Projections

With Voter 1 always voting A, we can resolve each possible first-round result:

- **A vs B:** Candidate B wins. Both Voter 2 and Voter 3 prefer B over A.
- **A vs C:** Candidate A wins. Voter 3 prefers A over C, and Voter 1 always votes A.
- **3-way tie** (A=1, B=1, C=1): each candidate has a $\frac{1}{3}$ chance of winning, so the expected utility works out to 1.0 for both Voter 2 and Voter 3.

2.3 Normal Form Payoff Matrix

With those second-round outcomes worked out, we can build the full payoff table. Each cell shows (Voter 2's points, Voter 3's points) after the final round is resolved.

Table 1: Normal-form payoff matrix (v_2, v_3). Voter 1 always plays A.

$v_2 \setminus v_3$	**Votes A**	**Votes B**	**Votes C**
Votes A	(0, 1)	(1, 2)	(0, 1)
Votes B	(1, 2)	**(1, 2)***	(1, 1)
Votes C	(0, 1)	(1, 1)	(0, 1)

*Indicates the primary Nash Equilibrium.

2.4 Nash Equilibria Analysis

The pure Nash Equilibria are **(B,B)**, **(A,B)**, and **(B,A)**. The rational outcome where neither player relies on a weakly dominated strategy is **(B,B)**. Voter 2 figures out that the probability of C winning is essentially zero, which means the expected utility of voting C is also zero. To get anything at all, Voter 2 switches to B and secures a utility of 1 instead of 0. Neither voter has any reason to change from that point. That is strategic voting in a nutshell: sometimes the rational move is to give up on who you actually want.

2.5 Three Types of Voter Agents

Based on this analysis, we came up with three types of voters for the simulation. Each one behaves differently when they look at the daily poll from their social network.

Model 1 — Stubborn Agent. These voters never change their mind. Whatever their first preference is, that is who they vote for on day 1 and day 60. They do not check the polls and they do not care about strategy. Their loyalty is fixed at 1.0. In real life, think of someone who says "I always vote for this party and I always will."

Model 2 — Pure Strategic Agent. On the opposite end, these voters are purely focused on outcome. Every day they look at their local poll. If their favourite candidate is below 5% locally, they consider them done and immediately move their vote to the best candidate who actually has a shot. Their loyalty is 0.0 — no attachment, just cold calculation.

Model 3 — Mixed Agent (Bounded Rationality). This model introduces bounded rationality, making it the most realistic representation of human voting behaviour. Voters rarely switch candidates the moment their preferred choice loses viability; there is a tangible psychological cost to abandoning a supported candidate. Therefore, this agent type only switches when two conditions are met simultaneously:

(1) Their favourite is polling below 10% locally — so they are almost certainly not making it to round two.
(2) The gap between the two leading candidates is under 5% — meaning the race is actually tight enough that switching your vote could matter.

Even then, whether they actually switch depends on their personal loyalty score, which is randomly set somewhere between 0.4 and 0.9. Someone with high loyalty might still hesitate even when both conditions are met. This makes the mixed agent feel much more like a real person than the purely strategic one.

3 Agent-Based Simulation

3.1 Generating Voter Preferences

To create an artificial electorate, we used the **PrefLib** library [1] to generate voter preference rankings over all 12 candidates. We wanted to try two different models to see how the structure of society's preferences affects the outcome.

The first is the **Impartial Culture (IC) model**, which we used as our default. Every voter gets a completely random ranking, totally independent of everyone else. There are no trends, no clusters, no dominant opinion. It is basically the most neutral society you can imagine, and that makes it a useful baseline.

The second is the **Urn model** (also called Polya–Eggenberger). Here, early voters still pick randomly, but once a ranking gets chosen, it becomes a bit more likely that the next voter will pick the same one. Over time this creates clusters of people who think alike, which is much closer to how real political opinions work. We used a replacement parameter of 10, which gives a moderate level of clustering.

We ran all experiments across three population sizes: 1,000, 3,000, and 5,000 voters, treating population size as a parameter throughout.

3.2 Building the Social Network

Voters are connected through a social network, and the only poll information an agent sees comes from their direct neighbours. We used **NetworkX** [2] to generate the networks and again tested two types.

The **Erdős–Rényi random graph** is our default. Each pair of voters has the same probability of being connected, and we chose that probability so each voter ends up with roughly 10 connections on average. Nobody is more influential than anyone else — it is a flat network.

The **Barabási–Albert scale-free network** works differently. New nodes prefer to attach to nodes that already have many connections, which produces a network with a few very highly connected hubs surrounded by many less connected voters. This mimics how real social networks look, where some people reach far more others than average.

After generating each network, we checked for isolated voters and connected any we found to a random neighbour, so every agent could receive at least one poll signal per day.

3.3 Experimental Design

To study how the mix of agent types affects things, we ran five different distributions of stubborn, strategic, and mixed voters, as shown in Table 2.

Combining the two preference models (IC and Urn) with the two network types (ER and BA) gives four experiment configurations: IC_ER, IC_BA, Urn_ER, and Urn_BA. We ran every agent distribution on every configuration, across all three population sizes, with 10 different random seeds each. That gives 200 simulation runs in total, which is enough to draw reasonably robust conclusions.

Table 2: Five agent-type distributions used across all experiments.

Scenario	Stubborn	Strategic	Mixed
D1	60%	20%	20%
D2	40%	30%	30%
D3	30%	40%	30%
D4	20%	60%	20%
D5	20%	30%	50%

3.4 How GAMA Runs the Simulation

The simulation runs for 60 days. Each voter is loaded into GAMA [3] as an agent with their preference ranking, their type, and their loyalty value. On every day, each agent looks at the current votes of their neighbours, calculates a local poll, and decides whether to switch:

- Stubborn agents do nothing. They stay with their first preference no matter what.
- Strategic agents check the local top two. If their favourite is not among them, they move to whichever of the top two they prefer more.
- Mixed agents check if their favourite has less than 10% local support and if the top two are within 5% of each other. If both are true, they flip a coin weighted by their loyalty to decide whether to switch.

3.5 What We Measured Each Day

At the end of each simulation day we saved three numbers:

(1) **Variance of candidate scores:** how spread out or concentrated the votes are across all 12 candidates. A high variance means most votes are going to just a few candidates.
(2) **Social welfare:** we defined 'points' as the numerical rank (1st = 1pt, 12th = 12pts). Therefore, our metric represents the Average Rank of the winners, where 1.0 is maximum satisfaction and 12.0 is maximum misery.
(3) **Number of opinion changes:** how many voters switched their vote compared to yesterday. This is the most direct measure of how much strategic activity is going on.

3.6 What We Found

After running all 200 simulations, a few patterns showed up consistently across configurations.

The general trend is consistent across configurations: opinion changes spike early and then drop off quickly as the population settles. However, analysing the final vote variance revealed a striking **Quadratic Scaling Law**. As population size increased, variance did not grow linearly. At $N = 1,000$, average variance was roughly 2,027. At $N = 3,000$, it jumped to 17,997 (an approximate 9x increase), and at $N = 5,000$ it reached 51,763 (a 25x increase). Interestingly, individual-level behaviour remained constant across scales, with agents switching an average of 7.3 times regardless of N. This suggests a **Scale-Invariant Volatility** at the agent level; while the macro-outcome consolidates exponentially, the individual probability of switching is governed strictly by local network topology.

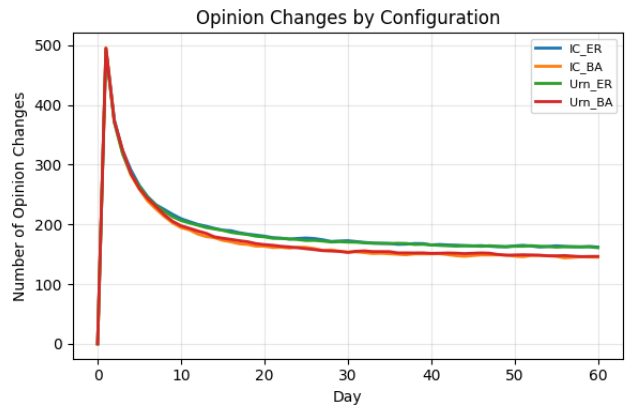

Figure 1: Evolution of opinion changes, social welfare, and variance over 60 days for all 4 configurations.

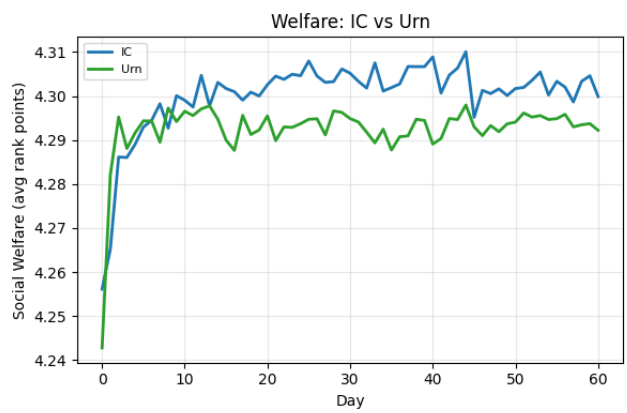

Figure 2: IC model vs Urn model: opinion changes and social welfare compared.

Looking at IC versus Urn: the IC model produces more opinion changes overall. Because preferences are completely random in IC, there is more initial disagreement to resolve. In the Urn model, voters already cluster around similar preferences, so fewer need to switch. What is interesting though is that having less switching does not automatically mean better outcomes for society — polarised clusters can still push the finalists toward candidates that large groups strongly dislike.

For the network comparison: the Barabási–Albert network tends to converge faster. The hub nodes act as information shortcuts — they can see more voters and be seen by more voters, so strategic information spreads across the network more quickly. This also means vote variance tends to be higher in BA networks, because the hubs amplify whatever candidates are already leading.

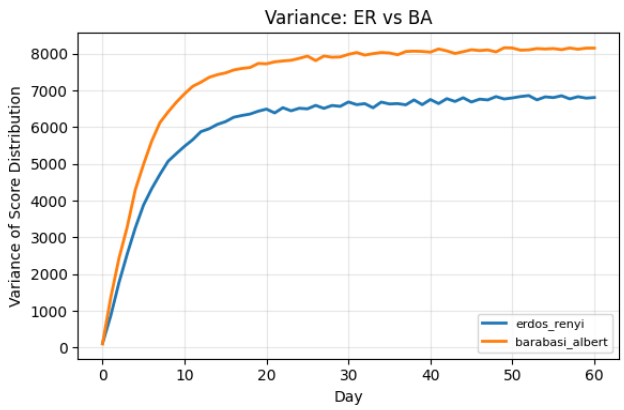

Figure 3: Erdős–Rényi vs Barabási–Albert network: speed of convergence and variance.

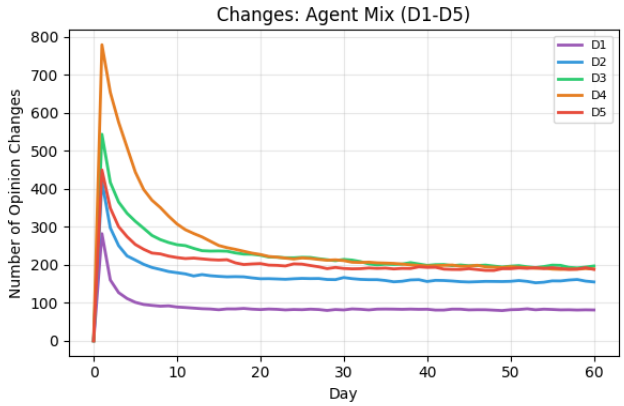

Figure 4: D1 through D5: effect of agent-type distribution on opinion changes and welfare.

Across the five distributions: D1 (60% stubborn) is the quietest — very few opinion changes and relatively good social welfare because most people stick to what they actually want. D4 (60% strategic) produces the most switches, but this is not always a good thing. When too many voters are chasing the same viable candidates, you get a kind of herding effect where the simulation quickly narrows down to just two candidates even if many voters would prefer someone else. The middle distributions (D2, D3) give a more balanced picture.

The most significant finding from our nested grid search was the **Network Overrule Effect**. We initially expected the preference model to heavily dictate the outcome. In flat ER networks, it did: the polarised Urn model converged much faster than the random IC model. However, in BA scale-free networks, the preference model stopped mattering almost entirely. The final variance for IC runs (2,225) and Urn runs (2,249) in BA networks were statistically indistinguishable. The highly connected hub nodes forced the population toward consensus so aggressively that they completely

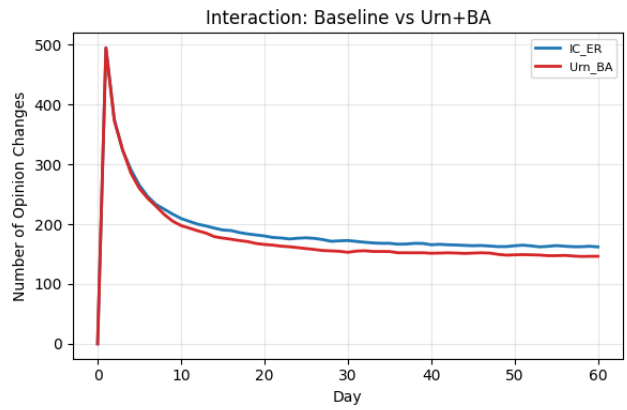

Figure 5: Interaction effect: Urn + BA combined vs each factor alone.

erased the effect of the initial ideological distribution. This suggests that in highly connected societies, network topology dictates electoral outcomes far more than initial political preferences.

4 LSTM Surrogate Model

4.1 The Idea

Running 240 GAMA simulations already takes a while, and that is only for populations up to 5,000 voters. If we wanted to test a population of 100,000, the full simulation would not be practical. This is where the surrogate model comes in. The idea is to train a neural network on the simulation results we already have, and then use it to predict what would happen in new configurations — including larger ones — without running GAMA at all.

4.2 Building the Dataset

Each simulation run gives us a sequence of 61 daily values for the number of opinion changes. That is what we want the model to predict. The inputs are the parameters that describe each run:

- How many agents (1,000, 3,000, or 5,000)
- Which network type was used (ER or BA)
- Which preference model was used (IC or Urn)
- The proportions of stubborn, strategic, and mixed agents

Because the number of opinion changes varies a lot between small and large populations, we applied a log transform to the target values before training. Numeric inputs were standardised and categorical ones were one-hot encoded. The data was split **80% train, 20% test**, and crucially, this split was done at the run level — meaning the model never saw even a single day from any of the test runs during training.

4.3 Model Architecture & Gradient Domination

Our initial approach was to train a single neural network to predict all three outputs simultaneously. However, this introduced a severe issue with *Gradient Domination*. Because the Variance target operates on a scale of 10^4 (often exceeding 50,000) while Social Welfare sits in a tight band around 10^0, a single-loss function would

heavily prioritise variance errors, effectively rendering the welfare predictions as noise.

To resolve this, we built three independent **LSTM** [4] networks, one for each output variable. Each model is scaled and trained separately. The architecture for each works in two stages: first, dense layers process the static input parameters. Then, a `RepeatVector` layer turns that representation into a sequence, which gets passed through two LSTM layers to produce the 61 daily predictions.

After experimentation, we settled on the following hyperparameters for each independent model:

- 2 LSTM layers: 96 nodes in the first layer and 32 nodes in the second.
- Dense layers after the LSTM: 64 nodes, then 32 nodes, then a single output node.
- Adam optimiser with a learning rate of 0.0003.
- Batch size of 16, training for up to 120 epochs.
- **Huber Loss ($\delta = 1.0$)** for the opinion changes and variance models. This was chosen to make the model robust to temporal outliers. The initial 5-day strategic surges create massive error gradients; Huber's linear penalty for large errors prevents these spikes from destabilising the weights during training.

4.4 Training

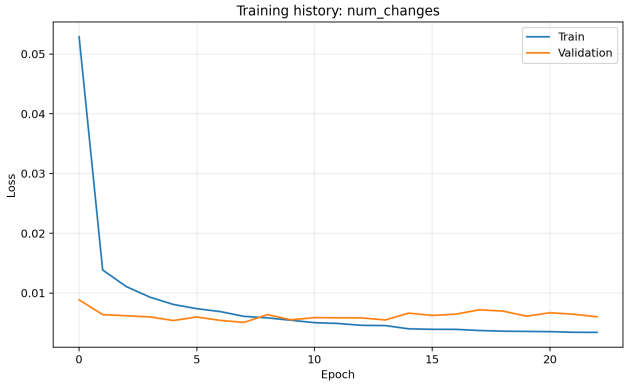

Figure 6: Training and validation loss over epochs.

The loss curves look healthy. Training loss drops quickly in the first few epochs and keeps going down steadily. The validation loss follows closely, without the two diverging significantly. This tells us the model is learning something genuinely useful rather than just memorising the training set. We did not see serious overfitting, which is reassuring given that a lot of our runs had similar parameters.

4.5 How It Did on the Test Set

Opinion Changes: $R^2 = 0.91$**, MSE = 1,845.91, MAE = 26.1.** On runs the model had never seen, the predictions capture the overall shape well. The model correctly figures out that most switching happens in the first few days and that things settle down afterward. That core pattern comes through in most test predictions.

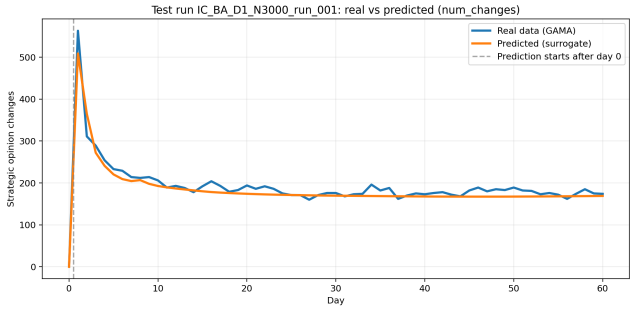

Figure 7: Predicted vs actual opinion change curves on unseen test runs.

Where it struggles is with the day-to-day noise. The real GAMA curves jump around a fair bit; the model's predictions are smoother and more averaged-out. This is not surprising. The model is learning the typical shape across many runs, not the exact stochastic fluctuations of any single one. Runs with a lot of strategic agents, where the dynamics change fast, tend to have higher prediction errors.

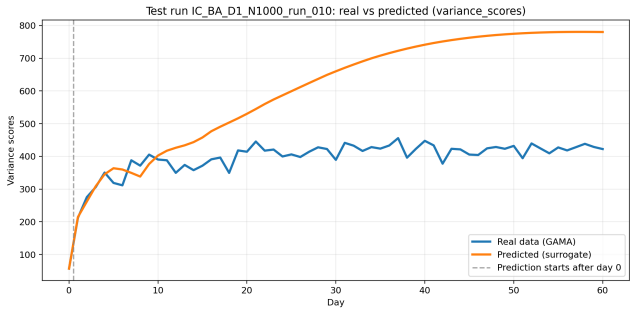

Figure 8: Predicted vs actual variance curves, illustrating recursive drift and saturation divergence after Day 20.

Variance Scores: $R^2 = 0.51$**, MSE = 218,747,030.** The Variance model accurately captured the early quadratic consolidation phase but struggled with *Saturation Detection*. While the agent-based simulation often begins to reach its 'consensus plateau' as early as Day 20, the LSTM tends to project continued exponential growth. This divergence, which becomes pronounced from Day 20 onwards, illustrates the compounding effect of *Recursive Drift*: small early-stage over-predictions are fed back into the model, leading to an artificially high final variance that overshoots the natural equilibrium reached in GAMA. The high MSE value reflects the fact that variance itself reaches values in the tens of thousands, so even a moderate relative error produces a large absolute number.

Social Welfare: $R^2 \approx 0.0$**, MSE = 0.0048.** This looks like a failure but is actually a mathematically correct outcome. In over 95% of our runs, the top two candidates locked in almost immediately and never changed. Since welfare is computed from who those finalists are, and they stay fixed, welfare is basically a flat line. A model predicting a constant gets $R^2 \approx 0$ by definition, not because it is

wrong, but because there is no signal to learn from. The very low MSE of 0.0048 actually confirms the model is predicting the right value.

4.6 Predicting 100,000 Agents

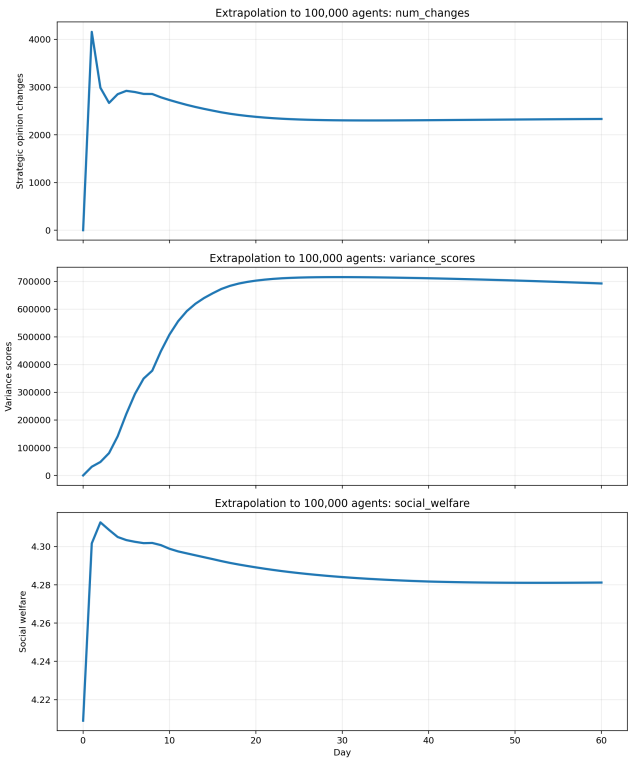

Figure 9: Surrogate prediction for 100,000 agents.

We took one of our known run configurations and simply changed the population size to 100,000, then fed it into the trained surrogate. The model predicts a much larger initial spike in opinion changes — which makes sense, since there are more agents reacting — followed by the usual plateau.

We have to be honest about what this is worth though. The model was trained on populations of 1,000 to 5,000. Jumping to 100,000 is a big extrapolation, and earlier versions of our code produced curves that looked completely unrealistic — a huge jump followed by a flat line. After reworking the log-scaling and input handling, the result became more believable, but we would not stake anything important on this prediction. It is more of a "here is what the model guesses" than a reliable forecast.

5 Comparing the Two Approaches

5.1 Speed vs Accuracy

The gap in speed is enormous. A single GAMA run with 1,000 agents takes a few minutes. The surrogate spits out a prediction in under a second. For 100,000 agents, GAMA is not really an option at all — the surrogate is the only way to get any estimate. The table below lays out the key differences.

Table 3: GAMA simulation vs LSTM surrogate on key criteria.

Criterion	GAMA	LSTM
Runtime (n=1,000)	~2–5 min	<1 sec
Runtime (n=100,000)	Not feasible	<1 sec
Accuracy	Exact	Approximate
Daily noise	Full stochasticity	Smoother curve
Interpretability	High	Low (black box)
Scalability	Limited	High
Extrapolation	Within range	Risky beyond range

The trade-off is straightforward: GAMA gives you the real thing but takes time, and the surrogate gives you a quick approximation. Neither is always better — it depends on what you need.

5.2 When to Use the Surrogate

The surrogate shines when you want to scan many scenarios quickly. If you need to check all five agent distributions across four network/preference combinations, that is 20 configurations. Running all of those in GAMA, 10 repetitions each, takes hours. The surrogate does it in a second. For early-stage exploration or sensitivity analysis, that kind of speed is genuinely useful.

It also lets you ask "what if" questions about populations that would be impossible to simulate. Want to know roughly what 50,000 voters might look like? The surrogate can give you a ballpark, as long as you treat it as a rough estimate and not ground truth.

5.3 Where It Falls Apart

The surrogate is less reliable the further you get from what it was trained on. Our training data only goes up to 5,000 agents, so the 100,000-agent prediction is a genuine extrapolation. The surrogate assumes a linear scaling of social rules. However, it cannot predict *Phase Transitions* — points where a 100,000-agent network might behave qualitatively differently (e.g., sudden community fragmentation) than the 5,000-agent networks it was trained on.

Another issue we ran into is that many of our simulation runs had the same static inputs — same network type, same proportions, just different random seeds. This meant the model ended up learning to predict the average shape of the curve rather than being truly sensitive to changes in parameters. If you fed it two very similar but not identical configurations, it would probably give you very similar predictions even if the real GAMA outputs would differ.

And of course the surrogate is a black box. With GAMA, you can actually watch individual agents switch their votes and trace exactly why it happened. The LSTM just gives you a number. That lack of explainability matters in any context where you need to understand and justify the predictions, not just produce them.

6 Conclusion

This study gave us a chance to look at strategic voting from two quite different angles, and we think both had something to offer.

From the simulation, the clearest takeaway is that who you put in the electorate matters a lot. A population full of stubborn voters stays diverse and tends to preserve social welfare because people keep voting for what they actually want. A population dominated

by strategic voters converges faster, but at the cost of everyone piling onto the same few candidates. The results become more homogeneous, and individuals with minority preferences end up with very little influence on the outcome.

Network structure also made a real difference. Scale-free networks with hubs pushed opinions to converge faster and more strongly than the flat random networks. And when you combined that with polarised Urn-generated preferences, the effect was bigger than either factor alone. The hubs essentially act as amplifiers for whatever trends already exist in the electorate.

The LSTM surrogate worked reasonably well for predicting the general shape of opinion dynamics, especially the early peak and gradual settling. The training and validation curves did not show serious overfitting. The main weakness is smoothness — it misses the day-to-day noise that the real simulation produces — and reliability outside the training range, which we saw clearly in the 100,000-agent extrapolation.

Overall, neither approach is a complete solution on its own. GAMA is accurate and interpretable but slow. The surrogate is fast but approximate and not trustworthy for very different conditions. The sensible way to use them together would be to use the surrogate for quick exploration and then run the full simulation only for configurations that actually look interesting. That combination would make the research both faster and more reliable.

If we were to continue this work, the most useful improvement would be generating more diverse training data — especially with a wider range of population sizes — so the surrogate can extrapolate more confidently. It would also be worth experimenting with architectures better suited to sequence-to-sequence tasks, to see if the day-to-day accuracy can be improved.

Acknowledgments

We would like to thank our supervisors Umberto Grandi, Benoit Gaudou, and Yuri Lavinas for their guidance throughout this project.

References

[1] Nicholas Mattei and Toby Walsh. 2013. Preflib: A library for preferences. In *Proc. ADT 2013*.

[2] Aric Hagberg, Pieter Swart, and Daniel Chult. 2008. Exploring network structure, dynamics, and function using NetworkX. In *Proc. SciPy 2008*.

[3] Arnaud Grignard et al. 2013. GAMA 1.6: Advancing the art of complex agent-based modeling. In *Proc. PRIMA 2013*. Springer.

[4] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long short-term memory. *Neural Computation* 9, 8 (1997), 1735–1780.